

Report related to the project developed during the course unit:

L.EIC018 - Laboratório de Computadores



Authored by:

Luís Costa (up202404078@up.pt)

Miguel Rocha (up202405484@up.pt)

Pedro Teixeira (up202404987@up.pt)

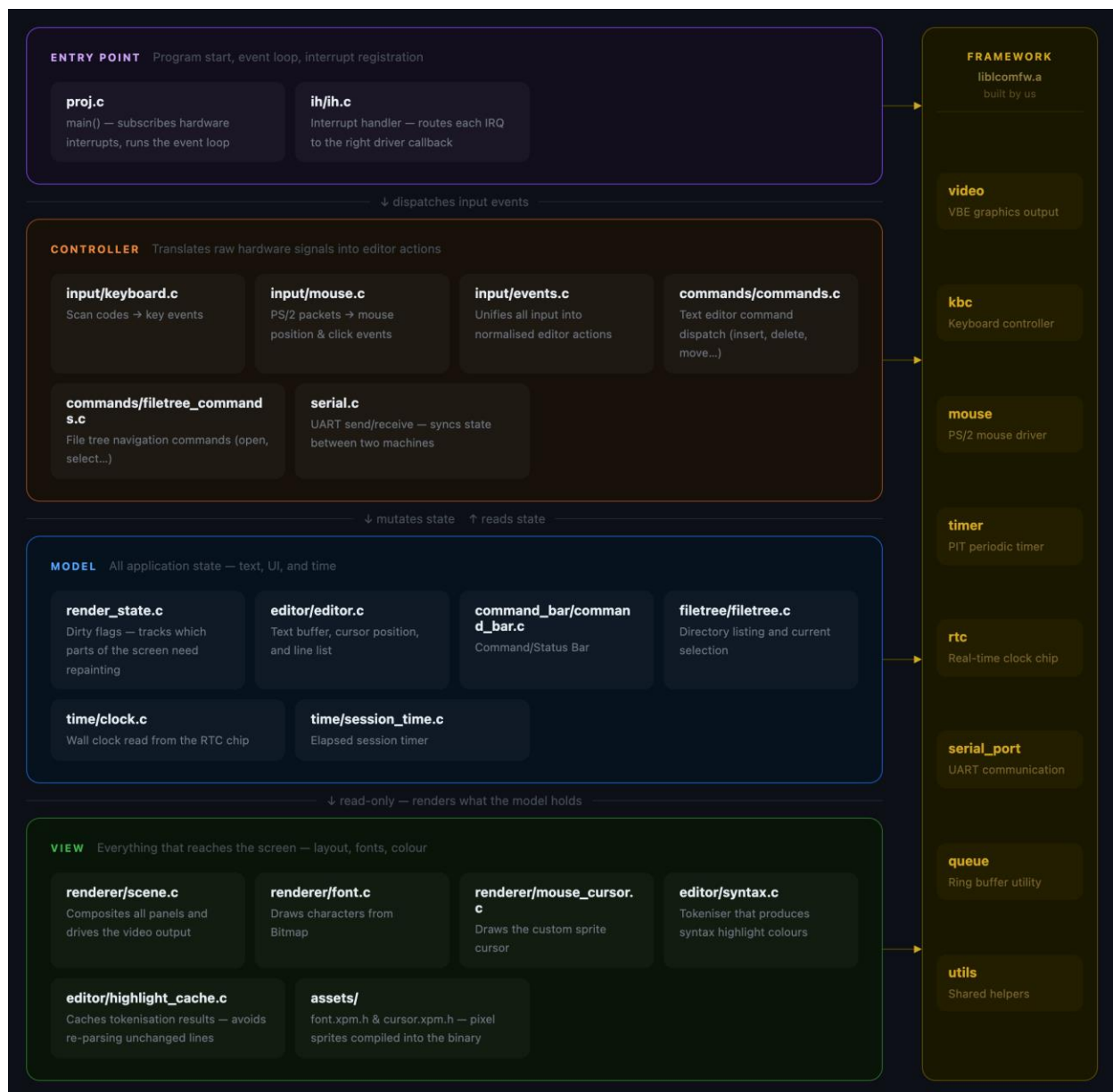
Rafael Silva (up202406334@up.pt)

LCode

What was our goal? What is our application?

During the project proposed for LCom we were tasked with developing a program that runs in Minix, using the drivers created throughout the class. Looking at Minix, and considering our personal ambitions and desire to learn how text/code editors work, we decided we wanted to try and create a Code Editor that runs in Minix, allowing users to edit files easily and developers to keep building inside the Minix environment.

How did we structure the project?



Our project is divided into two main parts: the framework and the application itself. The framework was built by reorganizing and extending the code developed throughout the laboratory assignments, providing the low-level functionality required by the project, including device drivers, interrupt handling, and hardware communication.

The application follows the Model–View–Controller (MVC) architecture shown in the figure above. The Model stores the editor state, the View is responsible for rendering the interface, and the Controller processes user input and translates it into actions that update the model.

What devices did we use, and for what purpose?

RTC (Real-Time Clock)

- Reads the current clock time (hours, minutes, seconds) to display a live clock in the status bar;

Timer

- Control's main loop tick-rate – `TIMER_HZ`;
- Control's command bar timed messages;
- Trigger's update every second to keep clock current;

Keyboard

- Translates scancodes to PT keyboard chars using keymaps (lowercase, uppercase (Shift) and AltGr);
- Handles nav keys (arrows, home, end) and special keys (enter, backspace, escape);

Mouse

- Tracks cursor position, clamping to screen size;
- Detects left cursor press and hold;
- Supports scroll wheel file navigation;

Video Card

- Used to display the multiple elements of the app: filetree; cursor; mouse pointer; status bar; and the file contents;

What are the differentiating features of our project?

Framework

A significant part of the project involved building a solid foundation before implementing editor features. Since we needed to integrate and extend the drivers developed throughout the labs, we invested time in improving their structure and creating a framework that simplified hardware interaction.

One of the framework's key improvements was the introduction of a backbuffer, a hidden screen that stores pixels before they are copied to the display. This approach reduces screen tearing and flickering while improving performance by minimizing direct graphics card operations.

As a result, the main project loop only needs to call framework functions, allowing development to focus on editor features rather than hardware management. We also introduced several quality-of-life utilities, such as a centralized fail() function that simplifies error handling and logging.

Development Environment

After creating the framework, we realized that rebuilding the framework and the project separately after every change was inefficient. To streamline development, we created run.sh, a shell script that automates the build process and allows either the framework or the full project to be executed with a single command.

Additionally, the project Makefile was adapted to automatically organize build artifacts, resulting in a cleaner and more efficient workflow.

FileTree Panel

Like most modern code editors, LCode includes a file tree panel that allows users to browse directories and open files directly from the editor. This feature greatly improves navigation and enables the development of larger, multi-file projects.

Rendering

Our initial implementation relied heavily on full-screen redraws, which caused unnecessary overhead even when only small parts of the interface changed. While functional, this approach led to performance issues during frequent updates, since most of the screen remained unchanged.

To improve efficiency, we introduced a structured set of rendering modes, allowing the system to update only the necessary regions. This includes partial redraws for elements such as the mouse cursor, individual characters, lines, words or regions, as well as the status bar and remotely modified lines. Full-screen rendering is now reserved only for cases where a complete refresh is required.

This incremental rendering approach significantly reduces redundant work and improves overall responsiveness.

Syntax Highlighting

LCode automatically detects a file's language from its extension and applies syntax highlighting accordingly. Currently, only C syntax highlighting is supported.

The editor performs an initial full-file parse to determine the color of each character, while subsequent updates only reprocess modified lines. To improve efficiency, we use caching mechanisms to store information such as whether a line belongs to a multi-line comment block. This avoids repeatedly scanning the entire file and provides a good balance between memory usage and performance.

Scroll-Wheel navigation

Early versions of the editor relied exclusively on keyboard navigation through arrow keys and Home/End commands. While sufficient for testing, this approach became impractical when working with larger source files.

To improve navigation, we first implemented a scrollbar. However, this still did not provide the level of responsiveness expected from a modern code editor when browsing large files. We therefore added support for the Microsoft IntelliMouse scroll wheel, which required extending our framework to correctly process and interpret scroll-wheel events.

The result is a much smoother and more intuitive navigation experience, allowing users to move through files efficiently without relying solely on keyboard shortcuts or scrollbar interactions.

Clipboard

As with most text editors, we wanted to support Ctrl+C and Ctrl+V, but implementing them proved more challenging than expected. In particular, handling multi-line selections that start or end in the middle of a line required careful management of partial line segments during copy and paste operations.

The difficulty was increased by our initial decision to store file contents as a character matrix. Ensuring correct behavior while avoiding memory leaks required considerable effort. Later, after transitioning to dynamic memory allocation, some of the existing safeguards allowed the implementation to be simplified. Despite the challenges, it was an important learning experience and a feature worth highlighting.

Serial Port

One of our goals was to make the editor collaborative by allowing two machines to edit the same file simultaneously through the serial port.

The first step was implementing a UART driver for COM1 configured at 115200 bps with hardware FIFOs enabled. Incoming data is handled through interrupts, which transfer bytes from the hardware queue into a software queue, while outgoing data is sent through polling.

A major challenge was defining what information should be transmitted. Sending a simple stream of characters was not enough, since the receiver also needs to know which operation to perform and where. To solve this, we designed a compact binary protocol consisting of a synchronization byte, a command byte, a payload length, and a variable-sized payload containing the command arguments.

The protocol supports operations such as character insertion and deletion, cursor movement, line updates, and larger modifications through block replacement commands. For complex actions like pasting or deleting multiple lines, only the affected data is transmitted instead of the entire file, reducing communication overhead.

To process incoming packets, we implemented a finite-state machine that reconstructs messages from the byte stream received through interrupts. Once a packet is complete, it is converted into an event and inserted into the same queue used by keyboard and mouse input. This allows remote edits to be handled transparently by the application.

Unfortunately, the communication is not fully operational due to a transmission block issue (LSR remains stuck at 0x00), preventing us from sending the successfully built packets, and we weren't able to figure out why. However, we implemented the core logic and believe it to be correct, as when testing via UART's loopback mode, packets are correctly transmitted, received, and successfully update the application's state.